

PROGRAMACIÓN APLICADA 2026-2

CAPÍTULO 06

Arquitectura Hexagonal aplicada al backend

Port IN · Use Case · Domain · Port OUT · Persistence Adapter

Documento	Guía del estudiante
Proyecto espejo	ParkFlow 360 · Cliente → Vehículo
Propósito	Transformar su primera entidad en un módulo hexagonal real siguiendo el proyecto espejo del profesor.

Principio central: el caso de uso no conoce JPA ni PostgreSQL.

1. Qué vas a construir

El profesor demostrará el patrón con ParkFlow 360 y la entidad Cliente. Tu grupo NO debe copiar Cliente. Debe aplicar exactamente la misma arquitectura a la entidad padre que seleccionó desde el Capítulo 01.

REGLA DE TRANSFERENCIA

Profesor: Cliente → Vehículo. Tu grupo: EntidadPadre → EntidadDependiente. En este capítulo trabajas solamente con tu EntidadPadre; la relación 1:N completa se retomará en el Capítulo 07.

2. Completa tu equivalencia antes de programar

Elemento del profesor	Tu proyecto
Proyecto	_____
Package base	com._____
Entidad padre	_____
Tabla PostgreSQL	_____
PK	_____
Campo UNIQUE 1	_____
Campo UNIQUE 2	_____
Schema	_____

3. Qué debes entender de arquitectura hexagonal

```

PETICIÓN HTTP
↓
ADAPTER IN (Controller)
↓
PORT IN
↓
APPLICATION / USE CASE
↓
DOMAIN
↓
PORT OUT
↓
ADAPTER OUT (Persistence)
↓
JPA / PostgreSQL

```

- Port IN = qué puede hacer el sistema.
- Use Case/Application = cómo se ejecuta esa capacidad.
- Domain = conceptos y reglas del negocio.

- Port OUT = qué necesita el núcleo del exterior.
- Adapter OUT = implementación concreta usando una tecnología.

4. Estructura de packages que debes crear

```
src/main/java/com/<tu-proyecto>/<tu-modulo-padre>
├── domain
│   ├── model
│   ├── exception
│   └── port
│       ├── in
│       └── out
├── application
│   └── service
├── infrastructure
│   └── adapter
│       ├── in
│       │   ├── web
│       │   │   ├── dto
│       │   │   └── mapper
│       └── out
│           ├── persistence
│           │   ├── entity
│           │   ├── mapper
│           └── repository
```

NO INVENTES NOMBRES AL AZAR

Los packages van en minúscula y expresan el módulo de negocio. Ejemplo: si tu entidad es Pedido, el módulo podría ser order o pedido según la convención acordada por tu equipo; mantén la decisión de forma consistente.

5. Paso 1 — Tu modelo de dominio

Crea una clase para tu entidad padre SIN @Entity y SIN @Table. Debe representar el negocio.

```
package com.<tu-proyecto>.<tu-modulo>.domain.model;

public class <EntidadPadre> {
    private Long id;
    private String <atributo1>;
    private String <atributo2>;
    private boolean activo;

    public <EntidadPadre>(Long id, String <atributo1>,
                        String <atributo2>, boolean activo) {
        this.id = id;
        this.<atributo1> = <atributo1>;
        this.<atributo2> = <atributo2>;
        this.activo = activo;
    }

    // getters
}
```

6. Paso 2 — Crea tu Port IN para registrar

```
package com.<tu-proyecto>.<tu-modulo>.domain.port.in;

import com.<tu-proyecto>.<tu-modulo>.domain.model.<EntidadPadre>;

public interface Registrar<EntidadPadre>UseCase {
    <EntidadPadre> registrar(<EntidadPadre> entidad);
}
```

7. Paso 3 — Crea tu Port IN para consultar

```
package com.<tu-proyecto>.<tu-modulo>.domain.port.in;

import java.util.Optional;
import com.<tu-proyecto>.<tu-modulo>.domain.model.<EntidadPadre>;

public interface Consultar<EntidadPadre>UseCase {
    Optional<<EntidadPadre>> buscarPorId(Long id);
}
```

8. Paso 4 — Crea tu Port OUT

Debes analizar tus constraints reales de PostgreSQL. Por cada UNIQUE importante, crea una operación de existencia si la necesitas como regla previa.

```
package com.<tu-proyecto>.<tu-modulo>.domain.port.out;

import java.util.Optional;
import com.<tu-proyecto>.<tu-modulo>.domain.model.<EntidadPadre>;

public interface <EntidadPadre>RepositoryPort {
    <EntidadPadre> guardar(<EntidadPadre> entidad);
    Optional<<EntidadPadre>> buscarPorId(Long id);
    boolean existePor<AtributoUnique>(String valor);
}
```

9. Paso 5 — Excepción de negocio

```
package com.<tu-proyecto>.<tu-modulo>.domain.exception;

public class <EntidadPadre>DuplicadaException extends RuntimeException {
    public <EntidadPadre>DuplicadaException(String message) {
        super(message);
    }
}
```

10. Paso 6 — Application Service

```
package com.<tu-proyecto>.<tu-modulo>.application.service;
```

```
import org.springframework.stereotype.Service;
import java.util.Optional;

@Service
public class <EntidadPadre>Service
    implements Registrar<EntidadPadre>UseCase,
        Consultar<EntidadPadre>UseCase {

    private final <EntidadPadre>RepositoryPort repositoryPort;

    public <EntidadPadre>Service(<EntidadPadre>RepositoryPort repositoryPort) {
        this.repositoryPort = repositoryPort;
    }

    @Override
    public <EntidadPadre> registrar(<EntidadPadre> entidad) {
        if (repositoryPort.existePor<AtributoUnique>(entidad.get<AtributoUnique>())) {
            throw new <EntidadPadre>DuplicadaException("Valor ya registrado");
        }
        return repositoryPort.guardar(entidad);
    }

    @Override
    public Optional<<EntidadPadre>> buscarPorId(Long id) {
        return repositoryPort.buscarPorId(id);
    }
}
```

NO COPIES SIN ENTENDER

Debes reemplazar todos los marcadores <...> por nombres reales de tu proyecto. Si tu tabla no tiene un UNIQUE equivalente, no inventes uno: usa las restricciones que realmente diseñó tu grupo.

11. Paso 7 — Tu entidad JPA

Aquí sí debes mapear tu tabla física real. Usa exactamente el schema, nombre de tabla, columnas, longitudes y nulabilidad de tu diseño.

```
@Entity
@Table(name = "<tabla_real>", schema = "<schema_real>")
public class <EntidadPadre>JpaEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "<pk_real>")
    private Long id;

    @Column(name = "<columna_real>", nullable = false, unique = true)
    private String <atributoUnique>;

    protected <EntidadPadre>JpaEntity() {}

    // resto de campos + getters/setters
}
```

12. Paso 8 — Spring Data Repository

```
public interface SpringData<EntidadPadre>Repository
```

```

        extends JpaRepository<<EntidadPadre>JpaEntity, Long> {

        boolean existsBy<AtributoUnique>(String valor);
    }

```

13. Paso 9 — Persistence Mapper

```

public final class <EntidadPadre>PersistenceMapper {

    private <EntidadPadre>PersistenceMapper() {}

    public static <EntidadPadre>JpaEntity toEntity(<EntidadPadre> domain) {
        // copiar dominio → JPA
    }

    public static <EntidadPadre> toDomain(<EntidadPadre>JpaEntity entity) {
        // copiar JPA → dominio
    }
}

```

14. Paso 10 — Persistence Adapter

```

@Component
public class <EntidadPadre>PersistenceAdapter
    implements <EntidadPadre>RepositoryPort {

    private final SpringData<EntidadPadre>Repository repository;

    public <EntidadPadre>PersistenceAdapter(
        SpringData<EntidadPadre>Repository repository) {
        this.repository = repository;
    }

    @Override
    public <EntidadPadre> guardar(<EntidadPadre> domain) {
        var entity = <EntidadPadre>PersistenceMapper.toEntity(domain);
        var saved = repository.save(entity);
        return <EntidadPadre>PersistenceMapper.toDomain(saved);
    }

    @Override
    public Optional<<EntidadPadre>> buscarPorId(Long id) {
        return repository.findById(id)
            .map(<EntidadPadre>PersistenceMapper::toDomain);
    }

    @Override
    public boolean existePor<AtributoUnique>(String valor) {
        return repository.existsBy<AtributoUnique>(valor);
    }
}

```

15. Paso 11 — DTOs y Web Mapper

```

public record Crear<EntidadPadre>Request(
    @NotBlank String <atributo1>,
    @NotBlank String <atributo2>
) {}

```

```
public record <EntidadPadre>Response(
    Long id,
    String <atributo1>,
    String <atributo2>,
    boolean activo
) {}
```

Tus validaciones deben venir del significado real de tus campos y de tu diseño físico. No copies @Email o @Pattern si tu atributo no los necesita.

16. Paso 12 — Controller dependiente de Port IN

```
@RestController
@RequestMapping("/api/<recurso-plural>")
public class <EntidadPadre>Controller {

    private final Registrar<EntidadPadre>UseCase registrar;
    private final Consultar<EntidadPadre>UseCase consultar;

    public <EntidadPadre>Controller(
        Registrar<EntidadPadre>UseCase registrar,
        Consultar<EntidadPadre>UseCase consultar) {
        this.registrar = registrar;
        this.consultar = consultar;
    }

    @PostMapping
    public ResponseEntity<<EntidadPadre>Response> crear(
        @Valid @RequestBody Crear<EntidadPadre>Request request) {
        // request → domain → use case → response
    }

    @GetMapping("/{id}")
    public ResponseEntity<<EntidadPadre>Response> buscar(@PathVariable Long id) {
        // use case → response
    }
}
```

17. Tus pruebas mínimas obligatorias

Prueba	Resultado esperado
POST válido	201 Created y fila real en PostgreSQL
GET con ID existente	200 OK
GET con ID inexistente	404 Not Found
POST violando validación HTTP	400 Bad Request
POST violando una regla UNIQUE	Error de negocio controlado / 409 cuando se implemente handler

18. Consulta de verificación en DataGrip

```
SELECT *
FROM <tu_schema>.<tu_tabla_padre>
ORDER BY <tu_pk>;
```

19. Checklist de arquitectura

- Mi Controller NO inyecta JpaRepository.
- Mi caso de uso depende de un Port OUT.
- Mi Port OUT NO extiende JpaRepository.
- Mi entidad de dominio NO tiene @Entity.
- Mi entidad JPA está dentro de infrastructure.
- Tengo mapper web y/o persistencia donde corresponde.
- Puedo señalar qué archivo pertenece a cada hexágono/frontera.
- Puedo registrar y consultar datos reales.

20. Preguntas para tu defensa individual

- ¿Cuál es tu Port IN y qué capacidad expresa?
- ¿Cuál es tu Port OUT y por qué está en el núcleo?
- ¿Qué clase implementa el Port OUT?
- ¿Qué clase conoce JpaRepository?
- ¿Qué clase conoce el nombre físico de la tabla?
- ¿Por qué tu dominio no tiene @Entity?
- Si cambiáramos PostgreSQL por otra tecnología, ¿qué piezas deberían cambiar?
- ¿Qué constraint de tu tabla estás respetando desde el caso de uso?

21. Evidencia que debes entregar

- Árbol de packages del módulo.
- Código de Port IN y Port OUT.
- Application Service.
- Persistence Adapter.
- POST exitoso.
- GET exitoso.
- Fila verificada en DataGrip.
- Commit Git identificable.

```
git add .
git commit -m "feat: aplicar arquitectura hexagonal a <entidad-padre>"
git push
```

22. Definition of Done

- Tu primera entidad funciona de extremo a extremo.
- La arquitectura puede dibujarse y explicarse.
- No existe acceso directo Controller → JpaRepository.
- La persistencia real sigue usando tu diseño físico existente.
- El código está versionado.
- Cualquier integrante puede defender la responsabilidad de cada package.

23. Qué viene en el Capítulo 07

Aplicaremos la misma arquitectura a tu entidad dependiente de la relación 1:N. El profesor usará Vehículo y la FK cliente_id. Tu equipo usará su tabla dependiente y su FK real. Aprenderás cómo manejar @ManyToOne sin permitir que la relación de persistencia invada el dominio.